American International University- Bangladesh (AIUB)
Faculty of Engineering (EEE)

Course Name:	Microprocessor and Embedded Systems	**Course Code:**	COE3104
Semester:	Summer 2024-2025	**Sec:**	K
Lab Instructor:	Md. Ali Noor		

Experiment No:	08
Experiment Name:	Implementation of a weather forecast system using the ADC modules of an Arduino.

Group Members	**Name**	**ID**
1.	Basudeb Kundu	23-50856-1
2.	Prohlad Chandra Das	23-50922-1
3.	Debashis Kumar Das	23-50953-1
4.	Indronill Dutta Nill	23-50974-1
5.	Nafiur Rahman Nirob	23-50991-1

Performance Date:	7-8-25	**Due Date:**	10-09-25

Marking Rubrics (to be filled by Lab Instructor)

Category	Proficient [6]	Good [4]	Acceptable [2]	Unacceptable [1]	Secured Marks
Theoretical Background, Methods & procedures sections	All information, measures and variables are provided and explained.	All Information provided that is sufficient, but more explanation is needed.	Most information correct, but some information may be missing or inaccurate.	Much information missing and/or inaccurate.	
Results	All of the criteria are met; results are described clearly and accurately;	Most criteria are met, but there may be some lack of clarity and/or incorrect information.	Experimental results don't match exactly with the theoretical values and/or analysis is unclear.	Experimental results are missing or incorrect;	
Discussion	Demonstrates thorough and sophisticated understanding. Conclusions drawn are appropriate for analyses;	Hypotheses are clearly stated, but some concluding statements not supported by data or data not well integrated.	Some hypotheses missing or misstated; conclusions not supported by data.	Conclusions don't match hypotheses, not supported by data; no integration of data from different sources.	
General formatting	Title page, placement of figures and figure captions, and other formatting issues all correct.	Minor errors in formatting.	Major errors and/or missing information.	Not proper style in text.	
Writing & organization	Writing is strong and easy to understand; ideas are fully elaborated and connected; effective transitions between sentences; no typographic, spelling, or grammatical errors.	Writing is clear and easy to understand; ideas are connected; effective transitions between sentences; minor typographic, spelling, or grammatical errors.	Most of the required criteria are met, but some lack of clarity, typographic, spelling, or grammatical errors are present.	Very unclear, many errors.	
Comments:				Total Marks (Out of):	

Experiment Title

Implementation of a weather forecast system using the ADC modules of an Arduino.

Objectives

The objectives of this experiment are to-

1. Familiarize the students with the Micro-controller-based weather forecast system
2. Measure environmental parameters, such as temperature, pressure, and humidity.

Theory and Methodology

Weather forecasting plays a critical role in various aspects of daily life, from agriculture and transportation to disaster preparedness. Accurate and timely weather predictions can help mitigate the risks associated with adverse weather conditions. With the advancement of embedded systems and sensor technology, it has become feasible to implement compact and cost-effective weather monitoring systems.

This experiment focuses on the development of a simple weather forecast system using an Arduino microcontroller and a barometric pressure sensor, specifically the BMP180 or MPL115A. These sensors are capable of measuring atmospheric pressure, which is a key indicator in predicting weather changes. The weather system interprets pressure data to forecast conditions such as sunny, cloudy, or rainy weather.

The implementation also includes an OLED display for real-time monitoring and makes use of the Analog-to-Digital Converter (ADC) modules of the Arduino to acquire environmental data. By analyzing the trend in barometric pressure over time, both simple and advanced prediction models can be realized. The simple model relies on direct pressure comparison, while the advanced model involves pressure gradient analysis over several hours for improved accuracy.

This experiment not only demonstrates the integration of hardware components and sensor data processing but also provides foundational insights into the working principles behind atmospheric pressure-based weather prediction.

Algorithms for weather Simple Approach:

Analysis	Output
dP > +0.25 kPa	Sun Symbol
-0.25 kPa < dP < 0.25 kPa	Sun/Cloud Symbol
dP < -0.25 kPa	Rain Symbol

Another approach that is more direct and quicker in calculating the weather in the simple approach is to know the current altitude. This cuts the need to wait and see a "trend".
By using the equation below:

$$ph = p0 \cdot e^{\frac{-h}{7990m}}$$

Simple Weather Station Code

```
Pweather = (101.3 * exp(((float)(CurrentAltitude))/(-7900)));

Simpleweatherdiff = decPcomp-Pweather;
if (Simpleweatherdiff >0.25)
  Simpleweatherstatus = 0; //Sun Symbol
if ((Simpleweatherdiff <=0.25) || (Simpleweatherdiff >=(-0.25)))
  Simpleweatherstatus = 1; //Sun/Cloud Symbol
if (Simpleweatherdiff <(-0.25))
Simpleweatherstatus = 2; //Rain Symbol
```

decPcomp (kPa) (MPL115A Pressure)	PWeather (kPa) (Ideal weather)	Simpleweatherdiff (kPa)	Weather Type
96.6	96.85	-0.25	Sun/Cloud
96.4	96.85	-0.45	Rain
97.4	96.85	0.55	Sun
96.92	96.85	0.07	Sun/Cloud

Advanced Version of Weather Station

Analysis	Output
dP/dt > 0.25 kPa/h	Quickly rising High-Pressure System, not stable
0.05 kPa/h < dP/dt < 0.25 kPa/h	Slowly rising High-Pressure System, stable good
-0.05 kPa/h < dP/dt < 0.05 kPa/h	Stable weather condition
-0.25 kPa/h < dP/dt < -0.05 kPa/h	Slowly falling Low-Pressure System, stable rainy
dP/dt < -0.25 kPa/h	Quickly falling Low Pressure, Thunderstorm, not

Apparatus

1) Arduino UNO board

2) BMP180/MPL115A

3) Inches96 inch OLED 128×64

4) Breadboard

5) Jumper Wires.

Circuit Diagram:

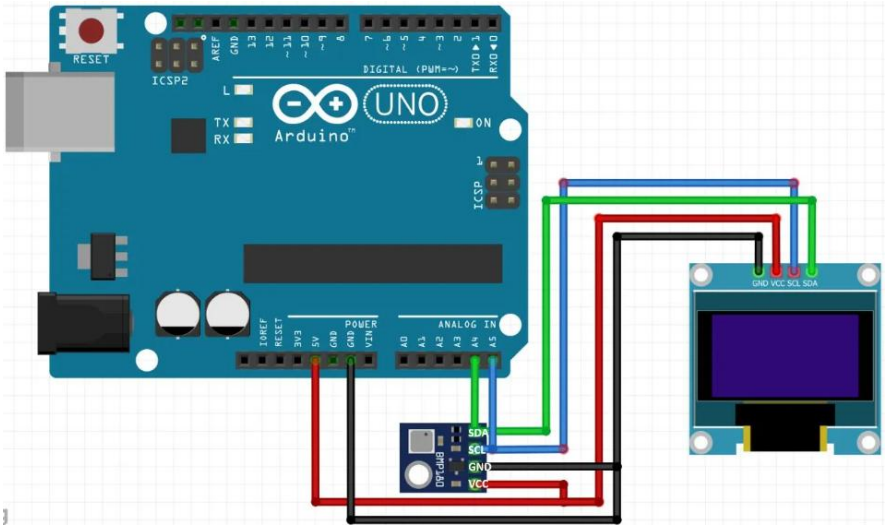

Fig1: Arduino Uno with BMP180 and OLED

Code/Program

The following is the code for the implementation of a weather forecast system using the ADC modules of an Arduino with the necessary code explanation:

```
    #include <SPI.h>
    #include <Wire.h>
    #include <Adafruit_GFX.h>
    #include <Adafruit_SSD1306.h>
    #include <Adafruit_BMP085.h>
    #define SCREEN_WIDTH 128
    #define SCREEN_HEIGHT 64

    Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT);
    Adafruit_BMP085 bmp;
    #define SEALEVELPRESSURE_HPA (101500)
    float simpleweatherdifference, currentpressure, predictedweather,
currentaltitude;
    void setup() {
    // put your setup code here, to run once:
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    if (!bmp.begin()) {
    Serial.println("Could not find a valid BMP085 sensor, check wiring!");
    while (1) {}
    }
    }

    void loop() {
    // put your main code here, to run repeatedly:
    display.clearDisplay();
    display.setTextSize(1);
```

```
  display.setTextColor(SSD1306_WHITE);

  display.setCursor(0,5);
  display.print("BMP180");
  display.setCursor(0,19);
  display.print("T=");
  display.print(bmp.readTemperature(),1);
  display.println("*C");

  /*prints BME180 pressure in Hectopascal Pressure Unit*/
  display.setCursor(0,30);
  display.print("P=");
  display.print(bmp.readPressure()/100.0F,1);
  display.println("hPa");

      /*prints BME180 altitude in meters*/
  display.setCursor(0,40);
  display.print("A=");
  display.print(bmp.readAltitude(SEALEVELPRESSURE_HPA),1);
  display.println("m");
  delay(6000);
  display.display();

  currentpressure=bmp.readPressure()/100.0;
  predictedweather=(101.3*exp(((float)(currentaltitude))/(-7900)));
  simpleweatherdifference=currentpressure-predictedweather;
  //display.clearDisplay();
  display.setCursor(0,50);
  if (simpleweatherdifference>0.25)
  display.print("SUNNY");

  if (simpleweatherdifference<=0.25)
  display.print("SUNNY/CLOUDY");

  if (simpleweatherdifference<-0.25)
  display.print("RAINY");
  display.display();
  delay(2000);
  }
```

Experimental Circuits And Result

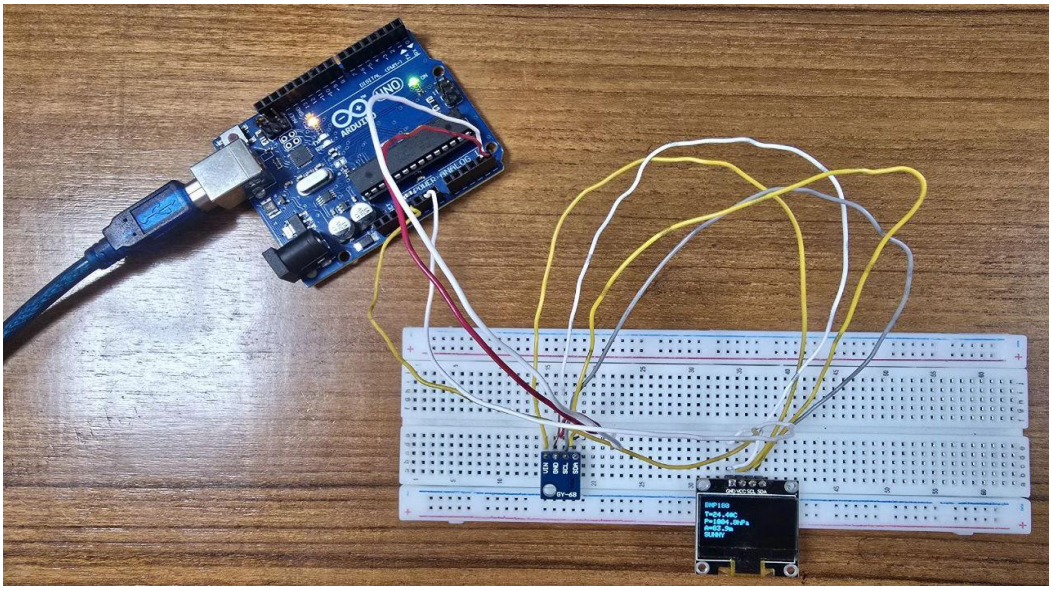

Fig2: Hardware implementation of a weather forecast system using the ADC modules of an Arduino

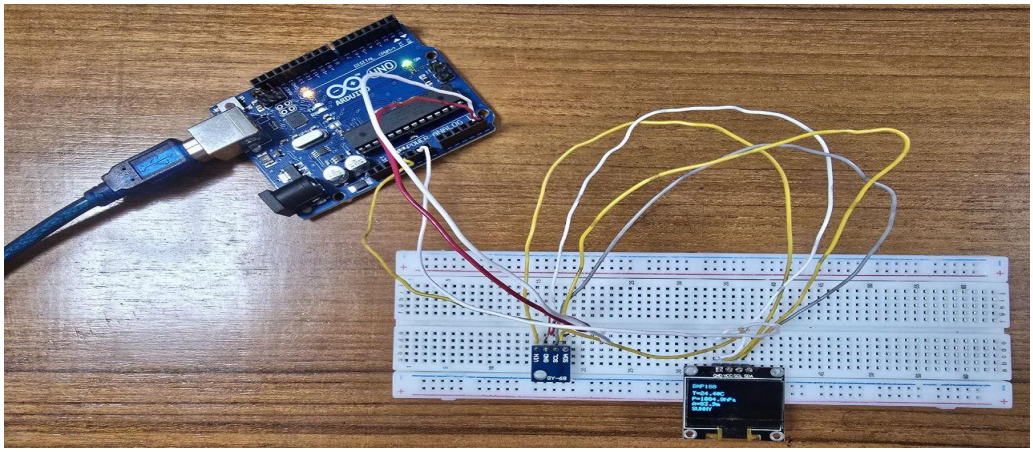

Fig3.1: First Readings where Temperature was 24.4°C, Altitude was 83.9m and SUNNY Weather

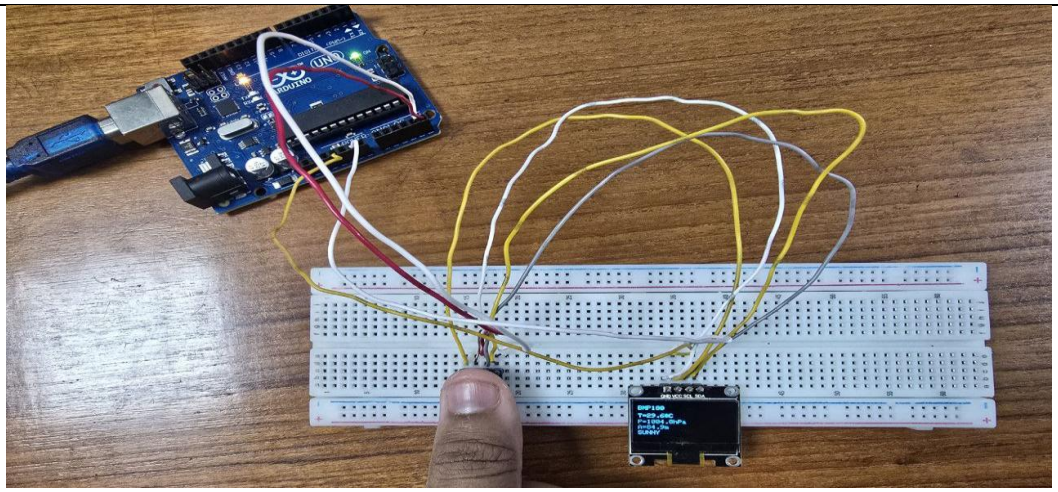

Fig3.2: Second Readings where Temperature was 29.6°C, Altitude was 83.9m and SUNNY Weather

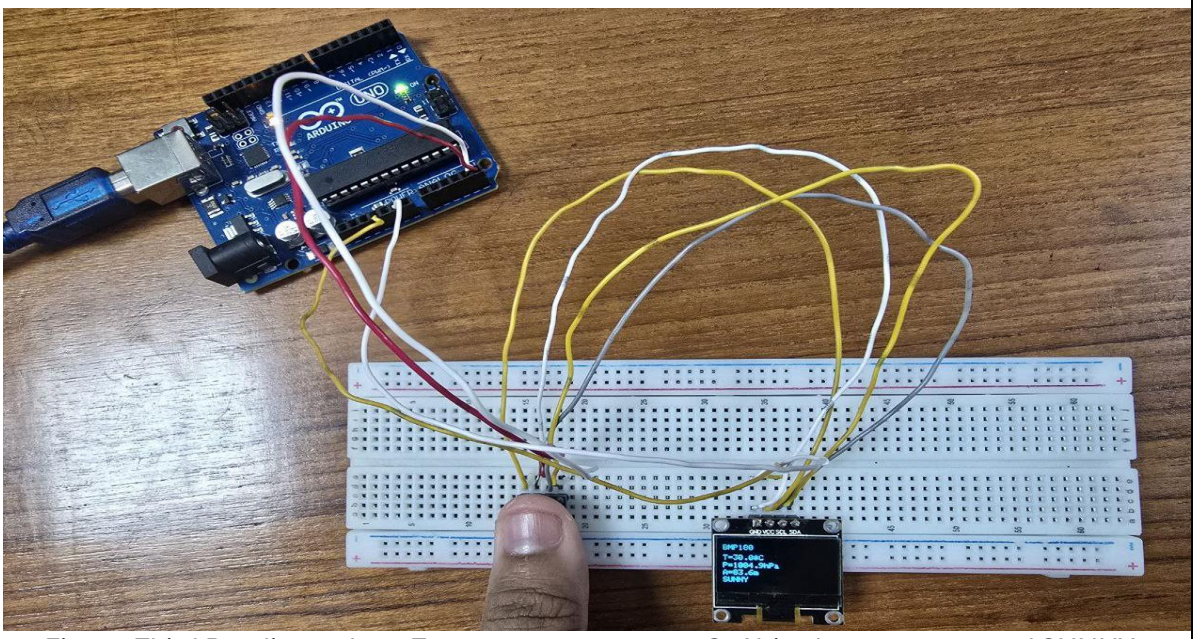

Fig3.3: Third Readings where Temperature was 29.6°C, Altitude was 83.9m and SUNNY Weather

Explanation:

The experimental circuit utilizes the BMP180 barometric pressure sensor connected to the Arduino UNO, along with a 0.96" OLED display. The sensor records temperature, pressure, and altitude, and based on the difference between the actual atmospheric pressure and the expected sea-level pressure (calculated using altitude), the system forecasts weather conditions.

From the data:

- **Fig 3.1** to **Fig 3.3** consistently showed a **temperature of around 29.6°C**, **altitude of 83.9m**, and **weather as SUNNY**.
  This suggests a steady high-pressure system, which aligns with the simple

algorithm rule:

```
dP > +0.25 kPa → Sunny.
```

Simulation Circuits And Result

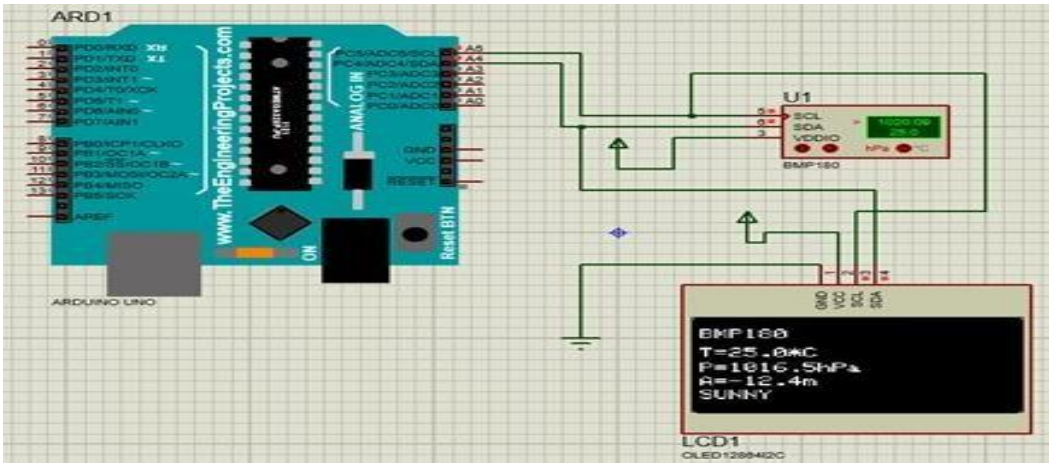

Fig4.1: First Readings where Temperature was 25°C, Altitude was 12.4m and SUNNY Weather

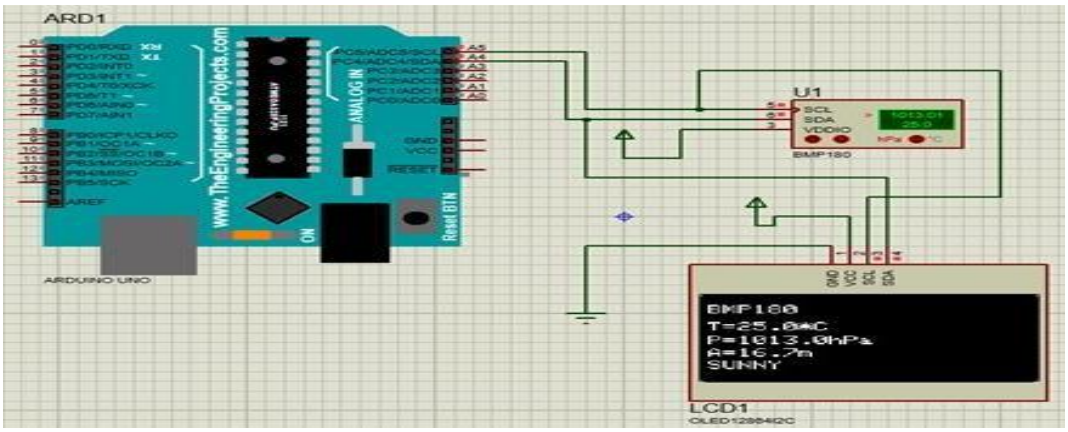

Fig4.2: Second Readings where Temperature was 25.0°C, Altitude was 16.7m and SUNNY Weather

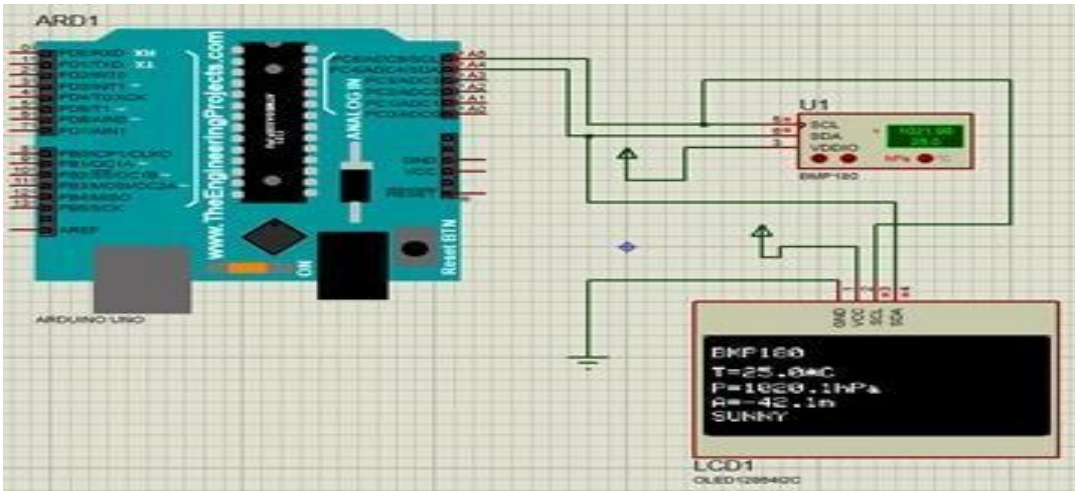

Fig4.3: Third Readings where Temperature was 25.0°C, Altitude was 42.1m and SUNNY Weather

Explanation:

The simulation environment replicates the same weather forecasting conditions. Across all readings (**Fig 4.1 to Fig 4.3**), the temperature remains constant at **25.0°C**, while the altitude varies slightly. The system consistently shows **SUNNY** weather, suggesting the pressure values remained above the +0.25 kPa threshold, simulating a stable high-pressure system in different altitudinal scenarios.

Discussion

The weather forecast system designed in this experiment demonstrates the effective use of a microcontroller (Arduino UNO) and barometric sensor (BMP180) to measure atmospheric pressure and predict weather conditions. The system successfully implemented a simple algorithm that correlates pressure differences with weather states such as sunny, cloudy, or rainy. The results from both the hardware implementation and simulation show consistent outputs under steady conditions, confirming the functionality and reliability of the sensor and algorithm within the tested environment.

However, the system does have limitations, particularly in its simplicity. The simple approach does not account for pressure changes over time, which are crucial for more accurate forecasting. Factors such as sudden weather changes, humidity, or real-time environmental influences are not considered, making the prediction basic and only suitable for short-term or static atmospheric conditions. Additionally, the experimental setup was conducted in a controlled indoor environment, which may not accurately reflect real-world outdoor variations.

To enhance the accuracy and utility of the system, future improvements could include incorporating the advanced forecasting method using pressure change over time (dP/dt), integrating additional sensors like humidity or temperature-humidity-pressure (THP) modules, and implementing data logging for long-term trend analysis. Furthermore, connecting the system to a cloud platform or mobile interface would allow remote monitoring and broaden its practical applications in fields such as agriculture, environmental monitoring, and smart home systems.

Reference

1. Adafruit BMP180 Sensor Datasheet – https://www.adafruit.com/product/1603
2. Arduino Official Documentation – https://www.arduino.cc/en/Guide
3. Adafruit GFX and SSD1306 Libraries – https://learn.adafruit.com
4. Barometric Pressure Forecasting – National Weather Service Educational Resources
5. "Practical Arduino Weather Station" – Marco Schwartz, 2015